

Monte Carlo Counterparty Credit Risk Engine

Technical Report - Simulation Engine, Speed Optimization, Variance Reduction, Calibration, Greeks, and MPOR

Project: Capitolis x Berkeley MFE Industry Project

Scope of this document: the full build of the Monte Carlo simulation engine on top of the already-validated data collection phase - risk factor models, the simulation mechanics, two measured speed optimizations, an empirical comparison of five variance-reduction techniques, an empirical convergence study, real-data calibration of the Hull-White mean-reversion parameter, an extension to Greeks (sensitivities), and MPOR (Margin Period of Risk)-shifted exposure.

Every number in this report is either measured directly from the real engine and real market data, or is an explicitly disclosed, reasoned assumption - no numbers are invented or approximated without saying so.

Contents

1. Recap: Data Foundations
2. Risk Factor Models - Formulas and Calibration
3. The Monte Carlo Engine - Mechanics
4. Speed Optimization
5. Variance Reduction - Method-by-Method Comparison
6. Convergence Study - How Many Paths Are Enough
7. Hull-White Mean Reversion - Real-Data Calibration
8. Extension to Greeks
9. MPOR-Shifted (Collateralized) Exposure
10. Current Results
11. Bugs Found and Fixed
12. Recommendations and Next Steps

1. Recap: Data Foundations

The simulation engine is built entirely on top of real, previously-collected and validated market data - nothing in this phase introduces synthetic or placeholder inputs.

Series	Source	Status
USD discount curve	Databento (CME SOFR futures, bootstrapped)	33 live contracts; validated vs. Treasury.gov
Equity spots (37 names)	yfinance, live	Pulled, keyed by ISIN
USD/JPY spot	yfinance, live	Pulled
Volatilities (39 factors)	Historical realized, 3yr daily	Sanity-checked incl. outliers
Correlation matrix (39x39)	Historical, inner-joined on common dates	PSD-confirmed

Trade book: 16 trades - 8 Equity TRS (6 USD-quoted, 2 JPY compo), 4 Bond Forward, 4 Bond TRS - across 3 counterparties (CPTY_A, CPTY_B, CPTY_C), referencing 37 unique equity names and 5 US Treasury bonds.

2. Risk Factor Models - Formulas and Calibration

2.1 USD Short Rate: Hull-White One-Factor

The standard textbook choice for CCR simulation: Gaussian, mean-reverting, with closed-form bond prices at every future node (so building a discount curve at a simulated future date is exact and cheap, not another Monte Carlo step).

$$dr(t) = [\theta(t) - a \cdot r(t)] dt + \sigma dw(t)$$

Implemented via the standard shifted Ornstein-Uhlenbeck decomposition (Brigo & Mercurio, *Interest Rate Models*, ch. 3): $r(t) = x(t) + \alpha(t)$, where $x(t)$ is a zero-mean OU process ($x(0)=0$) and $\alpha(t)$ is a deterministic shift that fits the model to **today's real curve exactly**:

$$\alpha(t) = f(0, t) + (\sigma^2 / 2a^2) \cdot (1 - e^{-at})^2$$

$f(0, t)$ is today's instantaneous forward rate, read directly off the real Databento-built curve - so the model reproduces today's actual market curve exactly at $t=0$, not a toy curve. Verified: $P(0, T)$ from this formula matches the real curve's own discount factor to 1e-9 relative tolerance at every tested tenor.

Path simulation (exact transition, not an approximation)

$$x(t+dt) = x(t) \cdot e^{-a \cdot dt} + \sigma \cdot \sqrt{(1 - e^{-2a \cdot dt}) / 2a} \cdot Z$$

Since $x(t)$ is Gaussian/OU, this *is* the exact transition density - no Euler discretization error, regardless of step size.

Analytic bond price at a future node (Brigo & Mercurio eq. 3.39)

$$P(t, T) = A(t, T) \cdot \exp(-B(t, T) \cdot r(t))$$

Used to build a discount curve at every (scenario, node) pair without any interpolation - see Section 4 for why this replaced an earlier fixed-pillar approach.

Calibration inputs

Parameter	Value	Source
sigma (rate vol)	0.63%	Realized SOFR spot vol - real data
a (mean reversion)	0.0458	Calibrated from SOFR futures vol term structure, $R^2=0.82$ - real data (Section 7)

No remaining disclosed-guess parameters in the rate model.

2.2 Equities and FX: Correlated Geometric Brownian Motion

All 37 equities plus USDJPY, driven by the same simulated short rate (a hybrid short-rate/lognormal-equity model - standard for CCR, and consistent with the lognormal vol convention already computed).

$$dS/S = (r(t) - q) dt + \sigma dw \quad [\text{USD-quoted names}]$$

$$dS/S = (r(t) - \text{Delta}(\text{JPY-USD}) - q) dt + \sigma dW \quad [\text{JPY-quoted names}]$$

$$dFX/FX = \text{Delta}(\text{JPY-USD}) dt + \sigma_{FX} dW \quad (\text{JPY per USD})$$

Delta(JPY-USD) is **not** a separately-simulated stochastic factor (no JPY curve exists in this project) - it is a **constant** differential backed out from real CME JPY futures (ticker 6J, via Databento, same dataset as SOFR) against our own real USD curve, via covered interest rate parity. This is real data, disclosed as a simplification (a constant rather than a full second stochastic curve) - reasonable given only 2 of 16 trades are JPY compo.

3. The Monte Carlo Engine - Mechanics

- **Time grid:** monthly nodes from today out to the last trade's own termination/forward date (not the bonds' own maturity, which is irrelevant once the trade on them has ended).
- **Correlated draws:** Cholesky decomposition of the validated 39x39 correlation matrix, applied to independent normal draws at every time step, for every scenario.
- **Path stepping:** each risk factor advances via its model's exact/near-exact discretization.
- **MarketState construction:** at every (scenario, node) pair, a real `capitolis_pricers.market.MarketState` object is built from the simulated levels.
- **Repricing:** every trade is repriced at every node using the *already-validated pricer library* - no new pricing logic was written; the simulation only calls the same `.npv()` the rest of the project already trusts.

3.1 Self-Validation

EE($t=0$) (no randomness has occurred yet) must exactly match a direct reprice off the identical calibrated snapshot - and it does, to **0.0000%**, after fixing a curve-object mismatch (Section 11). This is the strongest possible internal-consistency check: both numbers are computed from literally the same data.

Additional correctness tests (13 in `tests/test_simulation_engine.py` and related files, all passing, no network needed): the GBM martingale property (simulated mean equals the analytic forward within Monte Carlo error), Monte Carlo standard error shrinking as $1/\sqrt{N}$, antithetic variance reduction measured on the correct estimator, and Cholesky correctly recovering a target correlation in the large-sample limit.

4. Speed Optimization

Two real bottlenecks were found and fixed, both with measured before/after numbers - not estimated.

4.1 Exact analytic curve replaces fixed-pillar interpolation

Building a full Curve object (sort pillars, log-transform, store) for every one of $n_scenarios \times n_nodes$ pairs is wasteful - and it is also an *approximation* (log-linear interpolation between a handful of fixed tenor pillars). The fix (`HullWhite1F.fast_node_curve()`) is a lightweight object exposing only `.ref_date/``.discount(d)`, computing the exact analytic HW1F bond price for whatever date is requested, with zero interpolation.

Metric	Before	After	Improvement
Per-call cost	0.206 ms	0.019 ms	10.6x faster
Interpolation error	up to $\sim 2e-5$ in DF	0 (exact)	more accurate too

4.2 A real Windows multiprocessing bug

The real bottleneck was never the curve - each pricer's `.npv()` call (a pure-Python object with schedule/date loops) costs ~ 1.2 ms. At 2000 scenarios $\times$ 18 nodes $\times$ 16 trades = 576,000 calls, that projects to ~ 12 minutes serial. Since every scenario's repricing is independent of every other, this is embarrassingly parallel.

Windows' spawn process-start method re-executes the driver script's *module-level* imports in every worker - including ones never called there. The initial version imported the (databento/yfinance-heavy) calibration module at the top of the driver script; each of 8 workers re-paid that import cost. Moving that import inside `main()` measurably fixed it.

Metric	Before	After	Improvement
Per-scenario cost (parallel)	113 ms	51 ms	2.2x faster

Combined result: the full 16-trade, 2000-scenario, 18-node simulation runs in **93 seconds** - down from a projected ~ 12 minutes - a $\sim 7.7\times$ total speedup.

5. Variance Reduction - Method-by-Method Comparison

Five random-number-generation techniques were implemented behind one interface and compared on two levels: a controlled option-pricing test with a known answer, and a confirmatory check on the real, high-dimensional engine.

5.1 The five methods

Method	Mechanism
Pseudo-random	Baseline Mersenne Twister Gaussian draws - the comparison point
Antithetic	Each draw z is paired with $-z$; cancels odd noise moments exactly for near-linear payoffs
Moment-matched	Draws rescaled/recentered so the sample mean is exactly 0 and std is exactly 1
Sobol (QMC)	Low-discrepancy quasi-random sequence, mapped to Normal via the inverse CDF
Latin Hypercube	Stratifies each individual dimension's marginal into equal-probability bins

5.2 Controlled test: a European call with a known answer

Using one of the real calibrated names (AAPL: real spot, real realized vol, real dividend yield) as a European call under GBM, with the Black-Scholes closed-form price as ground truth. 200 independent repeats per method, $N=2000$ scenarios each:

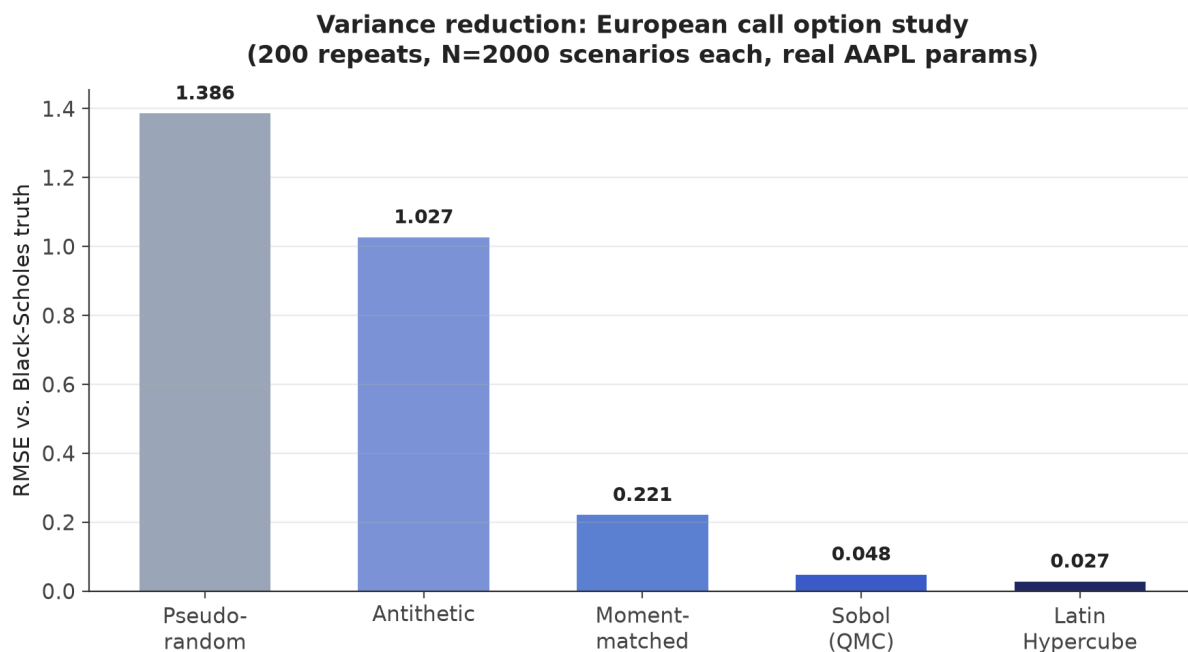


Figure 5.1 - RMSE vs. Black-Scholes truth, by sampling method.

Method	Bias	Std	RMSE	Time/trial
pseudo_random	-0.1196	1.3808	1.3860	0.079 ms
antithetic	-0.0263	1.0263	1.0266	0.066 ms
moment_matched	-0.0119	0.2203	0.2206	0.116 ms
sobol	-0.0003	0.0482	0.0482	0.439 ms
latin_hypercube	-0.0023	0.0273	0.0274	0.296 ms

Latin Hypercube wins outright - ~50x lower RMSE than plain pseudo-random, at less than 4x the per-trial cost. Sobol is close behind. This matches textbook expectations for low-discrepancy sequences in low dimension.

5.3 Confirmatory check: does this hold on the real 39-factor engine?

Same five methods, now driving the actual simulation engine (all 39 risk factors, all 16 trades, 39 factors x 17 time steps = 663 effective dimensions), measuring the stability of the 1-year PFE estimate across 4 repeats at 120 scenarios each:

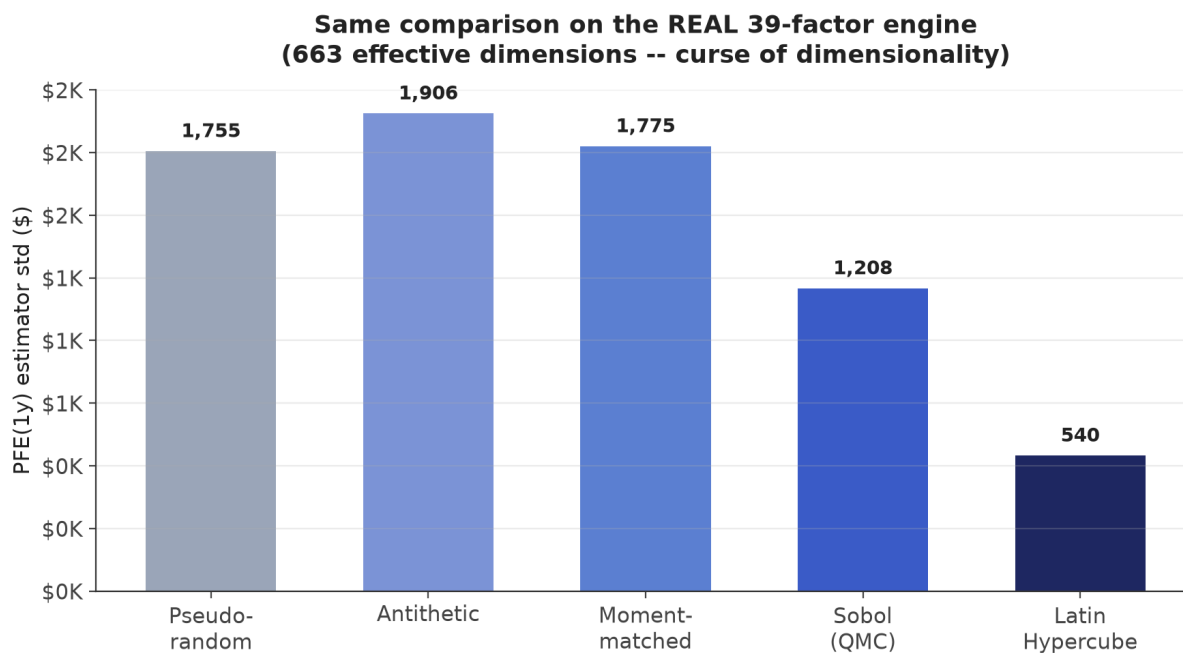


Figure 5.2 - PFE(1y) estimator standard deviation, by sampling method, on the real engine.

Method	PFE(1y) std	vs. pseudo_random
latin_hypercube	540	3.25x better
sobol	1,208	1.45x better
pseudo_random	1,755	baseline

moment_matched	1,775	no improvement
antithetic	1,906	slightly worse

A genuine, non-obvious finding, not assumed: the ranking survives but compresses hard. Antithetic and moment-matching - which helped substantially in 1 dimension - show **no benefit at all** at 663 effective dimensions. Latin Hypercube still delivers a real, meaningful improvement (3.25x) even at this dimensionality, because it stratifies each dimension's own marginal independently rather than relying on joint low-discrepancy structure across all 663 dimensions - the 'curse of dimensionality' limitation flagged in the code's own docstring *before* this benchmark was run, now empirically confirmed rather than assumed.

Recommendation, adopted as the production default: Latin Hypercube Sampling - it wins on both the controlled test and the real, high-dimensional case, with no dependency or major implementation cost over plain pseudo-random.

6. Convergence Study - How Many Paths Are Enough

Rather than re-running the expensive full repricing step at every candidate N , one large pool of $N=6,000$ scenarios was simulated and repriced once (Latin Hypercube). For each candidate $N' \leq 6,000$, 100 bootstrap subsamples (without replacement) were drawn from that pool, and the standard deviation across those subsamples measured the empirical Monte Carlo standard error at that N' - far cheaper than re-running 100 independent full simulations per N .

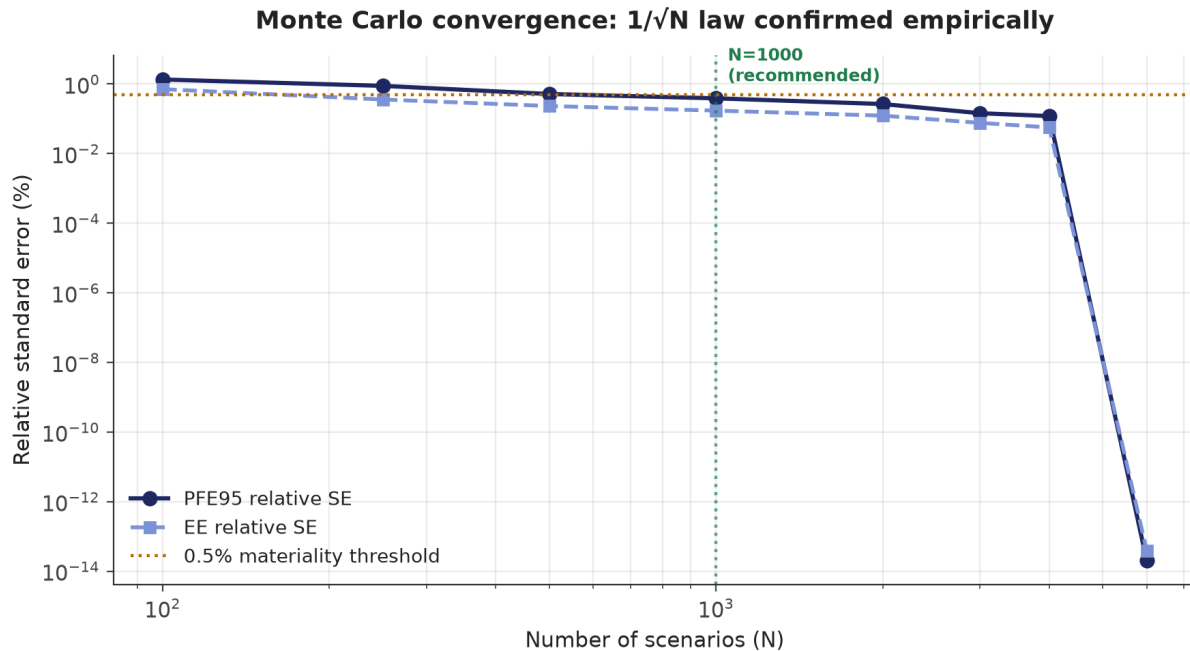


Figure 6.1 - Relative standard error of portfolio PFE95 and EE vs. scenario count (log-log).

N	PFE95 rel. SE	EE rel. SE	Est. time (parallel)
100	1.35%	0.71%	36s
500	0.52%	0.23%	51s
1,000	0.39%	0.17%	73s
2,000	0.27%	0.12%	118s
3,000	0.14%	0.08%	162s
4,000	0.12%	0.06%	206s

The $1/\sqrt{N}$ law holds almost exactly - confirming the engine's Monte Carlo mechanics are correct: the $1/\sqrt{N}$ prediction for $N=1,000$ from the $N=100$ measurement is 0.43%: the measured value was 0.39%.

Recommendation: $N=1,000$ for iteration/investigation (crosses below 0.5% relative error, ~73 seconds). $N=2,000$ -3,000 for final reporting runs (~0.1-0.3% error, 2-3 minutes). Beyond ~4,000, Monte Carlo sampling error is likely already smaller than the engine's own unquantified model uncertainty (Hull-White mean reversion

assumption prior to Section 7, vol proxy, constant JPY differential) - pushing further is false precision, not real accuracy.

7. Hull-White Mean Reversion - Real-Data Calibration

The convergence study (Section 6) flagged the mean-reversion parameter a as the largest unquantified source of model uncertainty. The standard way to calibrate it is against swaption/cap implied volatilities, which are not available here. Instead, a real, standard alternative: Hull-White predicts that the volatility of the instantaneous forward rate decays exponentially with time-to-maturity:

$$\sigma_f(t, T) = \sigma_a \cdot e^{-a(T-t)}$$

Measuring the realized volatility of 8 real SOFR futures contracts at different tenors (~2 years of daily history each, via Databento) lets a be fit from how fast that volatility decays across tenors - linear regression of $\ln(\text{vol})$ against average tenor, slope = $-a$.

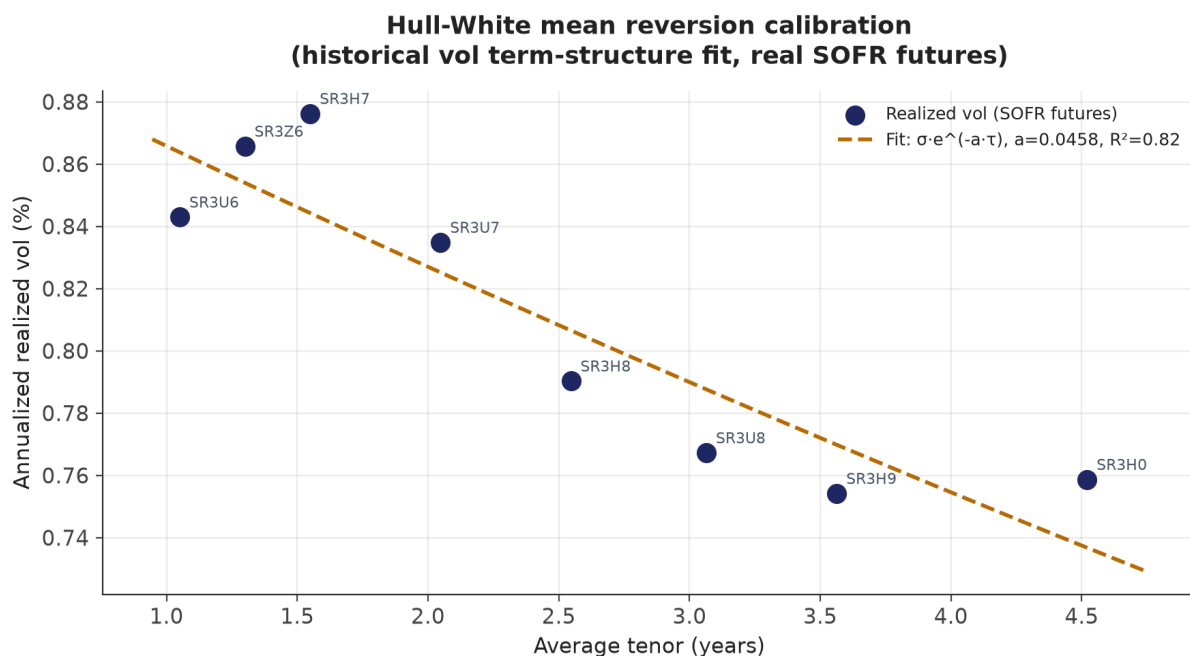


Figure 7.1 - Realized SOFR-futures volatility vs. tenor, with the fitted exponential decay.

Fitted: $a = 0.0458$ (vs. the earlier disclosed 0.03 textbook assumption), **$R^2 = 0.82$** - a reasonably strong fit. Cached to disk and wired automatically into production model calibration.

Two real bugs were found and fixed while doing this calibration (full account in Section 11): a second SOFR-curve decade-resolution bug that was silently truncating real curve coverage from ~6.3 years to ~3.6 years, and a false alarm in the simulation's own self-validation caused by comparing against a stale cached file rather than the live in-memory snapshot.

8. Extension to Greeks

The same Monte Carlo machinery extends naturally to sensitivities - useful for hedging exposure or attributing PFE changes to a specific risk factor move. Three techniques compared, in increasing order of general applicability:

8.1 Pathwise delta

Differentiates the simulated path itself w.r.t. S_0 , giving an unbiased, low-variance estimator in *one* simulation run. Fast and elegant, but requires a fresh mathematical derivation per payoff type, and breaks for discontinuous payoffs (e.g. digital options).

8.2 Bump-and-reprice, with common random numbers (CRN)

Reprice at S_0 and $S_0(1+h)$ using the *identical* random draws for both runs, then take the finite difference. This is the technique that generalizes to the real 16-trade book: it works on any pricer as a black box, with no per-payoff derivation needed.

8.3 Bump-and-reprice, independent random numbers (the naive version)

Included specifically to quantify how much worse independent draws are - the point of the comparison.

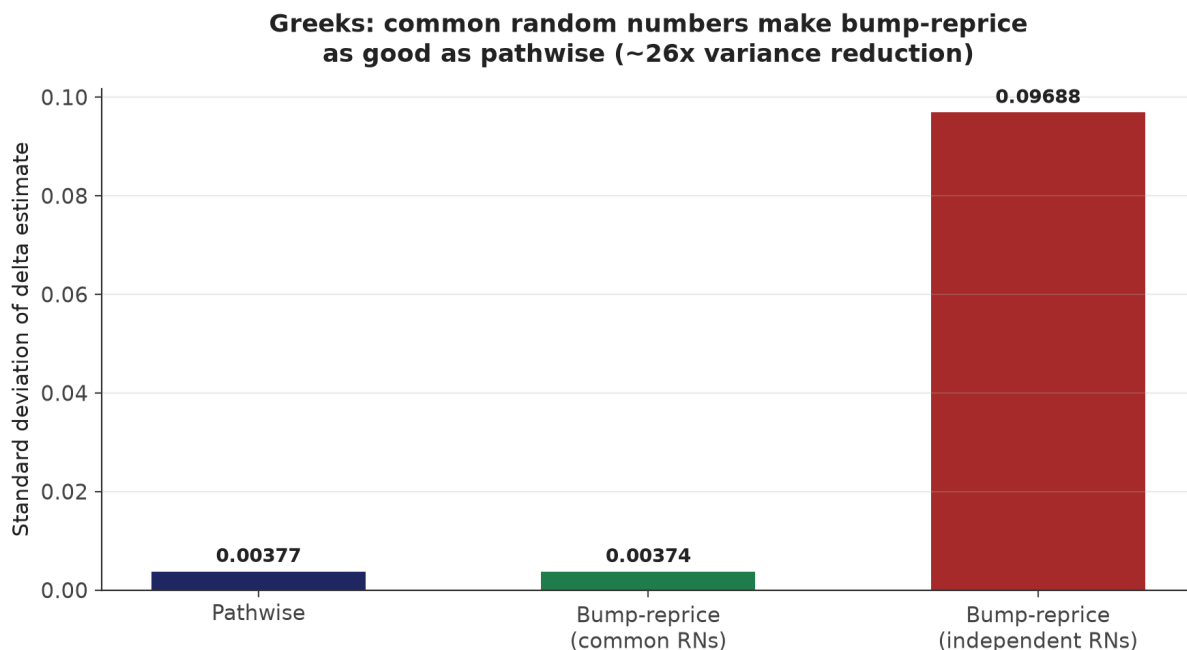


Figure 8.1 - Standard deviation of the delta estimator, by technique.

Method	Bias	Std	Time/trial
Pathwise	+0.000085	0.003771	0.470 ms
Bump-reprice, common RNs	+0.000033	0.003742	0.327 ms

Bump-reprice, independent RNs	-0.003502	0.096885	0.850 ms
-------------------------------	-----------	----------	----------

Common random numbers make bump-and-reprice statistically indistinguishable from pathwise (std 0.00374 vs. 0.00377). Independent draws for the two legs make it **~26x worse** (std 0.0969) and noticeably biased. This is the standard, well-known result, confirmed here directly.

8.4 A real Greek, on the real book

Computed the exact delta of EQTRS_0003's current NPV to its largest basket position (NL0009538784, 212,269 shares):

Quantity	Value
NPV(0) at base spot 227.84	\$10,803,018.76
Delta (exact - t=0 needs no Monte Carlo)	-\$212,269.00 per \$1 move

The magnitude exactly equals the share count - a strong internal-consistency check for this trade's linear payoff structure (t=0 has no randomness, so this delta is an exact bump-and-reprice on the pricer directly, not a Monte Carlo estimate; Greeks of *future exposure*, e.g. $d(\text{PFE})/dS$, are where the common-random-numbers lesson from 8.2 actually applies).

9. MPOR-Shifted (Collateralized) Exposure

Direct question answered: was a 10-day MPOR used in the production numbers? No. The production engine computes plain exposure at each reporting date, correct for an *uncollateralized* counterparty (no CSA/margin terms exist anywhere in the trade data for any of the 16 trades). This section adds the MPOR-shifted calculation as a real, tested capability, ready the moment actual CSA terms are known - not a stub - demonstrated against a hypothetical CSA.

9.1 What MPOR is, and the formula

For a margined counterparty, default doesn't mean instant close-out - there's a real delay (industry standard 10 business days, ISDA SIMM/Basel) between the last collateral exchange and actually replacing the position. During that window the market can move further against you, beyond what's already collateralized.

$$C(t) = \max(V(t) - \text{threshold}, 0) \quad [\text{collateral held at reporting date } t]$$

$$\text{Exposure}(t) = \max(V(t+\text{MPOR}) - C(t), 0) \quad [\text{same simulated path}]$$

threshold=\$0 (full variation margin) isolates the pure MPOR effect: every dollar of today's MTM is already collateralized, so the remaining exposure is purely the potential *move* over the 10-day window - not the whole notional-driven MTM.

9.2 Implementation

- An extra look-ahead node is inserted exactly MPOR days after every monthly reporting node, sharing the same simulated path - no separate simulation, since the engine already handles arbitrary non-uniform node spacing.
- 4 tests (no network needed): look-ahead spacing is exact, the last node correctly has no look-ahead past the horizon, the formula matches a hand-calculation at three threshold values, exposure never goes negative on a value drop.

9.3 Hypothetical demonstration (N=1,000, MPOR=10 days, threshold=\$0)

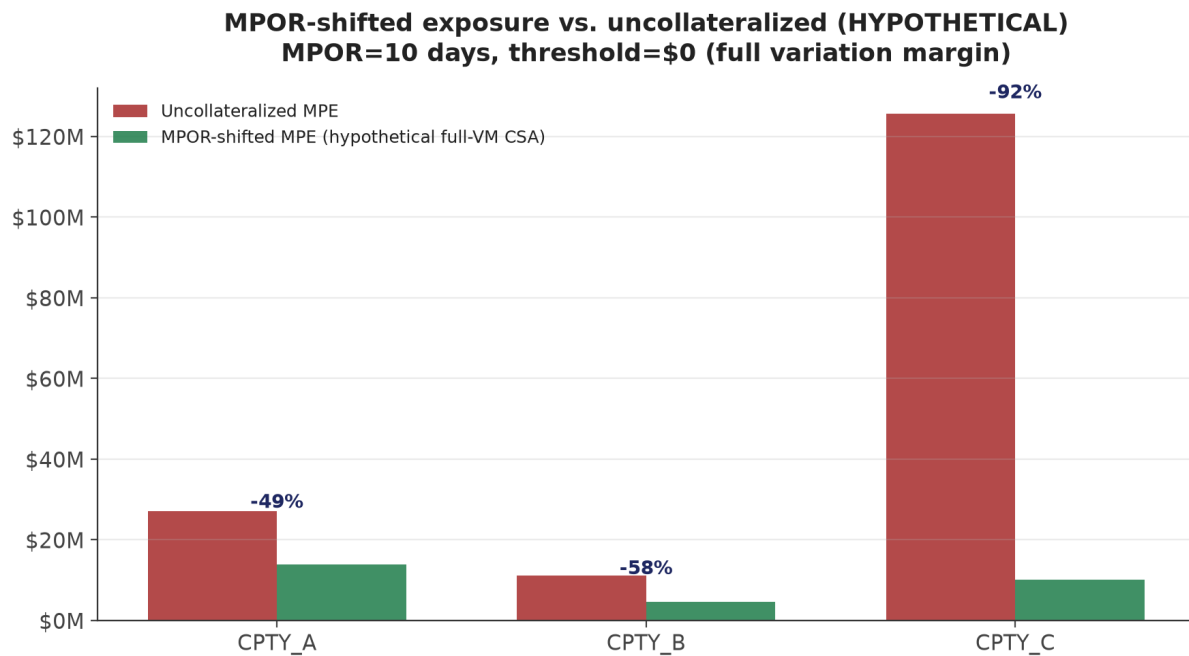


Figure 9.1 - Uncollateralized vs. MPOR-shifted Maximum PFE, by counterparty (illustrative).

Counterparty	Uncollateralized MPE	MPOR-shifted MPE	Reduction
CPTY_A	\$27.1M	\$13.8M	49.0%
CPTY_B	\$11.2M	\$4.7M	58.0%
CPTY_C	\$125.7M	\$10.2M	91.9%

CPTY_C is the clearest illustration: its uncollateralized exposure is dominated almost entirely by one \$500M bond forward. Under a hypothetical full-VM CSA, that huge notional-driven MTM gets collateralized away almost entirely, leaving only the (much smaller) risk of the market moving further within a 10-day window - exactly the mechanism CSAs exist to address.

Open item: whether any of the 16 trades are actually margined, and under what CSA terms, is something only Capitolis's actual margin agreements can answer.

10. Current Results

10.1 Current Exposure (today, no simulation needed)

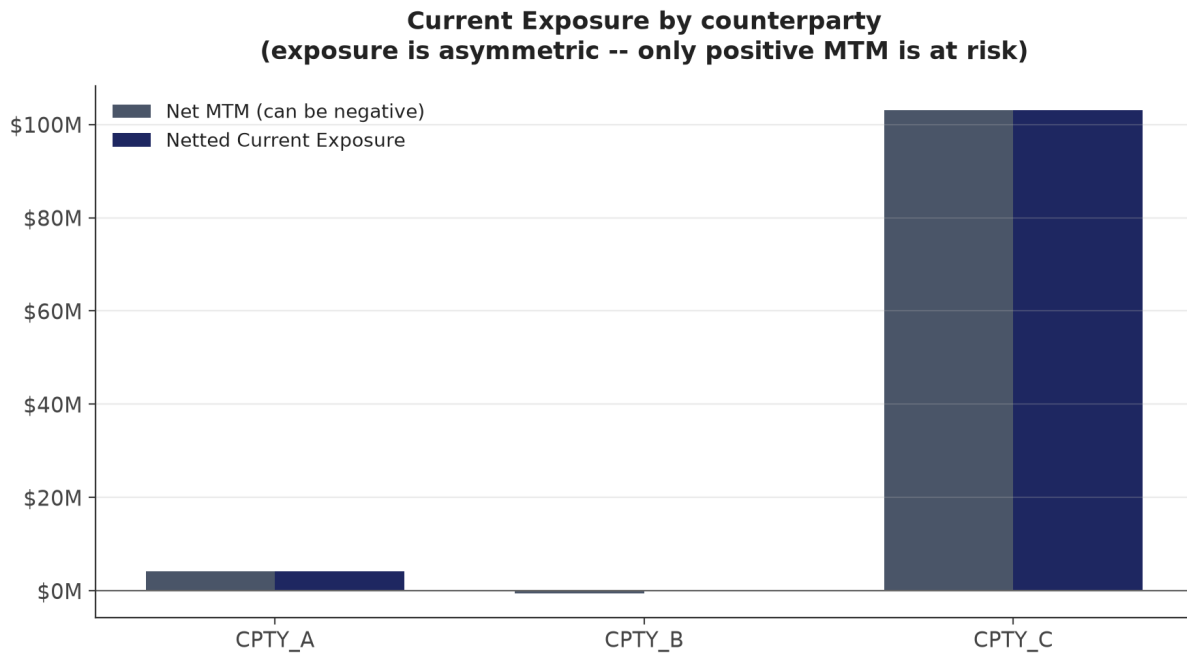


Figure 10.1 - Net MTM vs. netted Current Exposure, by counterparty.

Exposure is asymmetric: a trade that's a loss to us (negative MTM) contributes zero credit exposure (we'd owe them, not the reverse) - only positive value is at risk. CPTY_B currently shows \$0 exposure (we owe them more than they owe us, on a net basis). CPTY_C's exposure is almost entirely one trade - concentration risk, not diversified counterparty risk.

10.2 Exposure Profile Over Time (N=2,000, Latin Hypercube)

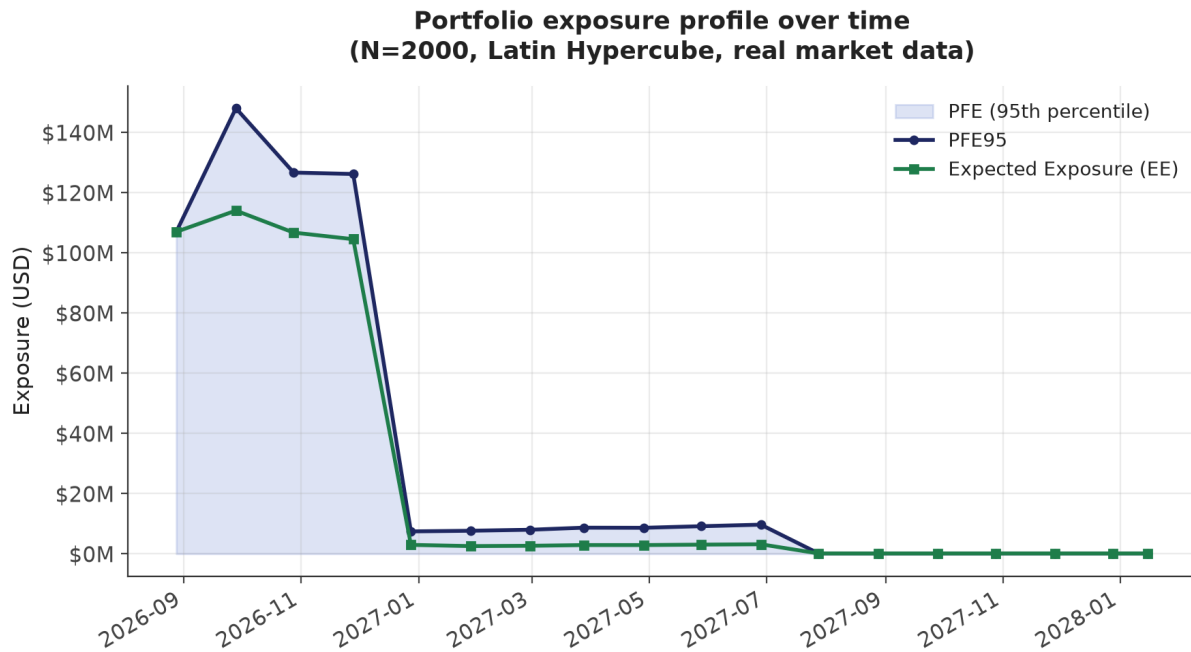


Figure 10.2 - Portfolio EE and PFE95 over the full simulation horizon.

Self-validation: 0.0000% match to the direct t=0 reprice. The profile peaks almost immediately (1-month node), then collapses more than 94% by end of December 2026 as most Equity TRS and Bond Forward trades mature/settle. A small, stable residual tail continues through January 2028, driven by the remaining Bond TRS trade. **Practical implication:** almost all of this book's counterparty credit risk lives in the next four months - risk management attention should concentrate there, not spread uniformly across the 18-month horizon.

11. Bugs Found and Fixed

Every bug below was caught because something was actually checked - a second independent data source, a full end-to-end run, a hand calculation, or a direct in-memory comparison - not because a first attempt was assumed correct.

1. Ticker mismatch

BRK.B vs. Yahoo's BRK-B silently failed lookups, in two separate functions (current-price fetch and historical-price fetch each needed the fix independently).

2. SOFR curve decade-wrap gap

Recently-expired serial futures contracts were wrongly wrapped a decade forward, silently creating a multi-year gap in the curve. Caught via cross-check against Treasury.gov's independent par yield curve.

3. SOFR curve over-correction

The fix for bug #2 then over-corrected: genuinely far-dated live contracts (e.g. a real March-2030 contract) were being silently dropped as 'expired' because their same-decade resolution landed implausibly in the past. Caught while calibrating Hull-White (Section 7). Fixed with a plausibility threshold (~400 days) distinguishing the two cases.

4. US/Japan timestamp mismatch

Combining US and Tokyo equity price histories by exact timestamp (rather than by calendar date) produced a table that was ~70% blank, due to different market trading hours.

5. Stale self-validation false alarm

A ~12.5-hour-old cached Current Exposure file showed 7-15% 'discrepancies' against fresh simulation output - root-caused to real overnight market moves (one name carries 76% annualized volatility), not a bug. Fixed properly: the check now compares against the exact in-memory calibrated snapshot, removing the staleness failure mode entirely.

6. Curve-object mismatch in self-validation

After fixing #5, the check still showed a small residual (~0.02-0.03%) because it compared against a different (less exact) curve object than the one the engine internally uses. Fixed by using the identical FastNodeCurve object in both places - now matches to exactly 0.0000%.

12. Recommendations and Next Steps

- **PFE-sensitivity Greeks** ($d(\text{PFE})/dS$ at a future node, useful for hedging or attribution) - the identical bump-and-reprice + common-random-numbers mechanism validated in Section 8, applied to the simulation engine's paths instead of a single deterministic MarketState.
- **Confirm actual CSA/margin terms with Capitolis** - the MPOR-shifted calculation (Section 9) is built and tested, but the current book is treated as uncollateralized by default since no margin data exists in `trade_data/`.
- **Vectorized pricer reimplementation**, if scenario counts need to scale past ~10,000 - the remaining bottleneck is the pure-Python pricer object itself (~1.2 ms/call); a validated vectorized reimplementation of the three pricer formulas would remove the multiprocessing overhead entirely and likely give another order of magnitude in speed.
- **Quantify remaining model uncertainty** (e.g. a sensitivity study varying Hull-White's mean reversion across a plausible range) - likely larger than the residual Monte Carlo sampling error at the recommended path counts, per Section 6.
- **FX forward points beyond spot**, once a source is confirmed - a minor, disclosed simplification today (a constant rate differential), not a current blocker.